ARM GICV3 / GICV4 · IHI 0069H.B · SENIOR CPU DV PREP

GIC Field Guide

The Generic Interrupt Controller takes interrupts from wires, messages and software, decides which processing element (PE) should take each one, ranks them by priority and security group, and hands the winner to the CPU. This guide covers it as an architect would explain it, a designer would build it and a verification lead would break it.

SGI · software generated PPI · private peripheral SPI · shared peripheral

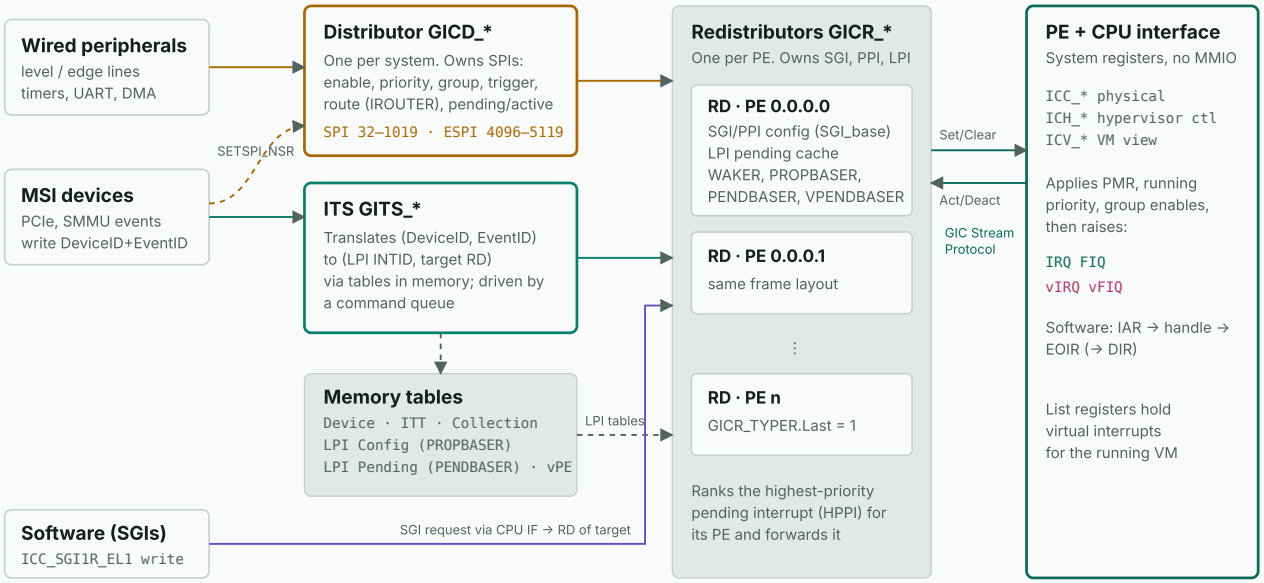
LPI · message based Virtual

---

01 · Architecture

Four blocks, one pipeline

GICv3 splits the old monolithic controller into blocks that scale to hundreds of cores. Keep this picture in your head; nearly every interview question maps onto one arrow in it.



The Distributor and ITS are shared. Redistributors and CPU interfaces are per PE. The Redistributor ↔ CPU interface link (GIC Stream Protocol, Appendix A) is the natural cut point for block-level verification.

What changed from GICv2

- Affinity routing (Aff3.Aff2.Aff1.Aff0) replaces the 8-bit CPU target mask, so the 8-core limit is gone.
- CPU interface moved to system registers (`ICC_*`), which is faster than MMIO.
- LPIs and the ITS add message-signaled interrupts.
- Two Security states with three groups, not just Group 0/1.

Version ladder

- **v3.0**: affinity routing, LPIs, ITS, sysreg CPU IF
- **v3.1**: Extended SPI/PPI ranges, MPAM, Secure EL2
- **v3.2**: Armv8-R AArch64
- **v3.3**: non-maskable interrupts (NMI)
- **v4.0**: direct injection of vLPIs
- **v4.1**: direct vSGIs, reworked vPE tables
- **v4.2**: virtual NMIs

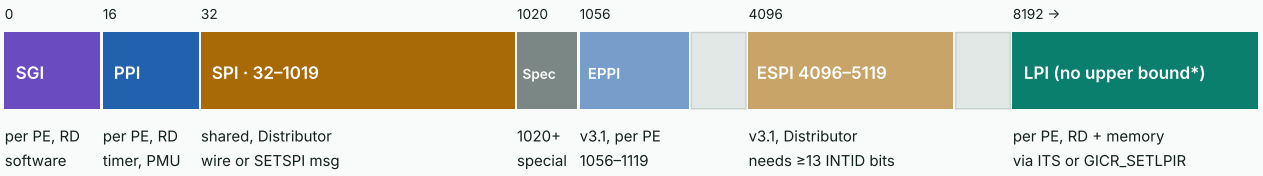
Spec map (IHI 0069H.b)

- Ch 2 routing · Ch 3 partitioning
- Ch 4 physical handling & priority
- Ch 5 LPIs & ITS · Ch 6 virtualization
- Ch 7–8 GICv4.0 / v4.1
- Ch 11 power · Ch 12 programmers' model
- Ch 14 system errors · App A stream protocol
- App B pseudocode (great for ref models)

02 · Specification

The INTID map

The interrupt ID (INTID) alone tells you the type, who owns its state and which block configures it. Interviewers expect these boundaries without hesitation.



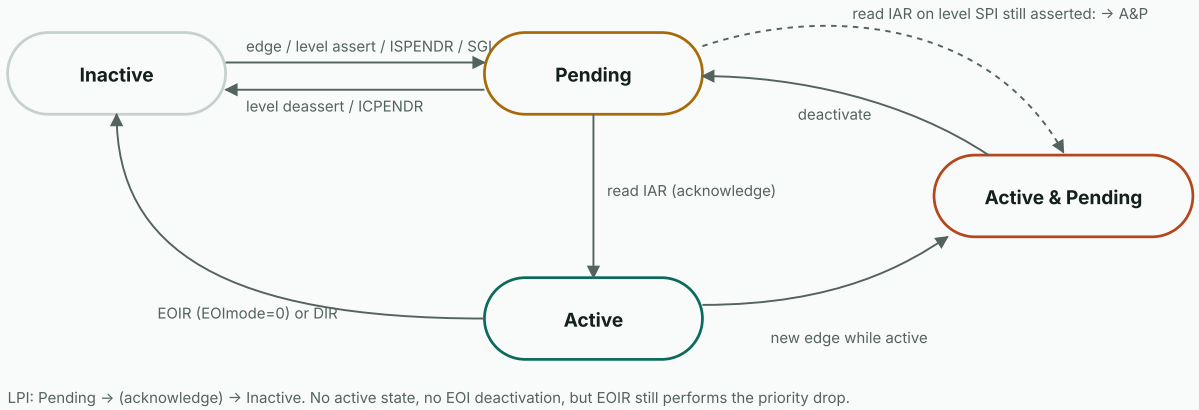
Schematic, not to scale. Grey gaps are reserved (1024–1055, 1120–4095, 5120–8191). *LPI ceiling set by GICD_TYPER.IDbits and GICR_PROPBASER.IDbits.

SPECIAL INTID	MEANING WHEN READ FROM IAR/HPPIR
1020	EL3 read of `ICC_IAR0_EL1`: the pending interrupt is Secure Group 1, meant for Secure EL1/EL2.
1021	EL3 read of `ICC_IAR0_EL1`: the pending interrupt is Non-secure Group 1, meant for NS EL1/EL2.
1022	GICv3.3: read of `ICC_IAR1_EL1` with `SCTLR_ELx.NMI`=1 when the HPPI is an NMI. Use `ICC_NMIAR1_EL1` to take it.
1023	Spurious: nothing pending has enough priority, or the HPPI does not suit the current Security state / exception level / register used.

03 · Specification

Interrupt lifecycle

Every SGI, PPI and SPI moves through four states. LPIs have only two: they have no Active state, so acknowledging an LPI clears its pending bit and nothing needs deactivating.



LPI: Pending → (acknowledge) → Inactive. No active state, no EOI deactivation, but EOIR still performs the priority drop.

Edge vs level. A level interrupt that stays asserted after acknowledge goes straight to Active & Pending. Software must clear the source before EOI, or it fires again immediately. An edge that arrives while Active becomes Active & Pending and is taken again after deactivation, so no edge is lost (but two edges while pending merge into one).

Interview trap. "Can an LPI be active?" No. "Does an LPI need EOI?" Yes, for the priority drop. With `EOImode` =1, a write to `ICC_DIR_EL1` for an LPI has no effect.

04 · Architecture

Groups and security

Each interrupt is in one of three groups, set by `IGROUPR` together with `IGRPMODR`. Whether it arrives as IRQ or FIQ depends on the group and on the Security state and EL the PE is in right now. This is the most common whiteboard question.

GROUP	IGROUPR / IGRPMODR	TYPICAL OWNER	PE IN SECURE EL1/EL2	PE IN NON-SECURE	PE AT EL3
Group 0	`0 / 0`	EL3 firmware	**FIQ**	**FIQ**	**FIQ**
Secure Group 1	`0 / 1`	Secure OS / TEE	**IRQ**	**FIQ**	**FIQ**
Non-secure Group 1	`1 / 0`	Rich OS / hypervisor	**FIQ**	**IRQ**	**FIQ**

Rule of thumb

An interrupt for the world you are in arrives as IRQ. An interrupt for another world arrives as FIQ, which routes to EL3 (`SCR_EL3.FIQ`=1) so the monitor can switch worlds.

Register banking

Non-secure accesses to Secure-group state read as zero and ignore writes. `GICD_CTLR` has separate `ARE_S` / `ARE_NS` and `EnableGrp0/1S/1NS` bits. `DS`=1 collapses to a single Security state.

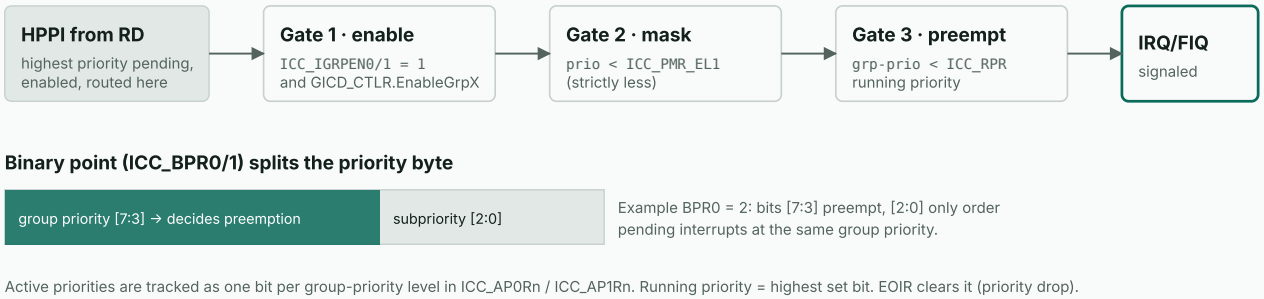
Priority split

Non-secure software sees a shifted priority view: its value `p` is stored as `(p>>1)|0x80`. NS interrupts therefore always occupy the lower-priority half, so Secure can always preempt.

05 · Specification

Priority, masking and preemption

Priority is 8 bits and lower means more urgent. A GIC with two Security states implements at least 32 levels (5 bits); one with a single state, at least 16. The CPU interface signals an interrupt only if it passes three gates.



Binary point (ICC_BPR0/1) splits the priority byte

Example BPR0 = 2: bits [7:3] preempt, [2:0] only order pending interrupts at the same group priority.

Active priorities are tracked as one bit per group-priority level in ICC_AP0Rn / ICC_AP1Rn. Running priority = highest set bit. EOIR clears it (priority drop).

The acknowledge / end sequence

- **Acknowledge**: read `ICC_IAR1_EL1`. Returns INTID, Pending→Active, sets active-priority bit, raises running priority.
- **Priority drop**: write `ICC_EOIR1_EL1`. Clears the highest active-priority bit.
- **Deactivate**: if `ICC_CTLR.EOImode` =0, the same EOIR write deactivates. If =1, a later write to `ICC_DIR_EL1` deactivates.
- EOIs must happen in reverse order of acknowledges (nested). DIR can be done in any order.

Why split EOImode?

Priority drop re-opens the CPU to new interrupts quickly, while the source stays Active so it cannot re-fire. Linux uses this for threaded handlers. Hypervisors use it so that the VM's virtual EOI performs the physical deactivation through a hardware-linked list register (HW=1).

NMI (v3.3)

An NMI is presented with superpriority. It ignores PMR and running-priority masking unless `PSTATE.ALLINT` masks it. Acknowledge with `ICC_NMIAR1_EL1`.

Routing and SGIs

SPI routing with affinity (ARE=1)

- `GICD_IROUTER<n>` holds a full Aff3.Aff2.Aff1.Aff0 target.
- `Interrupt_Routing_Mode` (IRM)=0 sends to that exact PE. IRM=1 sends to **any one** participating PE (1-of-N); the choice is IMPLEMENTATION DEFINED.
- A PE participates in 1-of-N only if its group enable is set, `GICR_CTLR.DPG*` does not exclude it, and it is awake.
- Changing routing while an SPI is pending: the spec allows either old or new target for in-flight signaling. Your scoreboard must allow both.

SGI generation

- Write `ICC_SGI1R_EL1` (NS G1 / own group), `ICC_ASGI1R_EL1` (the other Security state's G1) or `ICC_SGI0R_EL1` (G0).
- Fields: INTID[3:0], Aff3/2/1, TargetList (16 bits of Aff0), RS range selector (TargetList covers Aff0 = RS×16 + bit), IRM.
- IRM=1 broadcasts to every PE **except the sender**.
- Delivery is not synchronous with the write. Software needs a DSB plus ISB to order it, and the TB must not assume a fixed latency.

> **Legacy mode.** ARE=0 brings back GICv2-style 8-bit targets and memory-mapped CPU interface (Chapter 15). It is deprecated, and removed when Secure EL2 is implemented. Expect a question on why ARE must be programmed before anything else and why changing it with groups enabled is UNPREDICTABLE.

07 · Architecture + design

LPIs and the Interrupt Translation Service

LPIs scale to tens of thousands of interrupts because their configuration lives in memory, not flops. The Redistributor caches it. The ITS turns a device's MSI write into an LPI on the right core.



At the Redistributor

`LPI Configuration table` (GICR_PROPBASER, shared): 1 byte per LPI = priority[7:2] + enable[0]. Cached; change needs INV/INVALL.

`LPI Pending table` (GICR_PENDBASER, per RD): 1 bit per INTID. Only written back fully when RD is quiesced (EnableLPIs=0, power off).

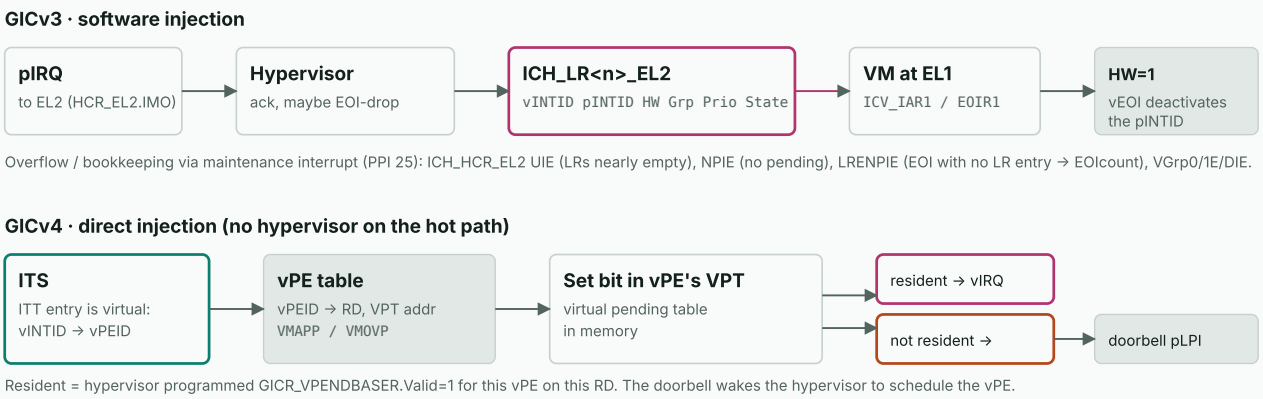
`Command queue`: ring buffer at GITS_CBASER, 32-byte commands. SW writes commands, then advances GITS_CWRITER. ITS advances GITS_CREADR as it consumes. Empty when CREADR == CWRITER.

On a command error the ITS may stall (GITS_CREADR.Stalled).

COMMAND	ARGUMENTS	WHAT IT DOES, AND THE DV ANGLE
`MAPD`	`DeviceID, ITT_addr, Size, V`	Map or unmap a device's ITT. Unmap with live mappings or pending LPIs is a classic corner.
`MAPC`	`ICID, RDbase, V`	Point a collection at a Redistributor. RDbase is a PA or a processor number, per `GITS_TYPER.PTA`.
`MAPTI / MAPI`	`DeviceID, EventID, [pINTID], ICID`	Create the event → LPI mapping (MAPI uses pINTID = EventID).
`MOVI`	`DeviceID, EventID, ICID`	Retarget one LPI. A pending instance must move with it, never be lost or duplicated.
`MOVALL`	`RDbase1, RDbase2`	Move all pending LPIs between RDs, used for CPU hotplug.
`INT / CLEAR`	`DeviceID, EventID`	Set or clear pending by command, as if the device wrote. Great TB stimulus.
`DISCARD`	`DeviceID, EventID`	Remove the mapping and clear pending state.
`INV / INVALL`	`DeviceID, EventID / ICID`	Make RD re-read config bytes from memory after software edits them.
`SYNC`	`RDbase`	Barrier: all earlier physical-LPI effects on that RD are visible before SYNC completes.
`VMAPP / VMAPTI / VMOVI / VMOVP / VSYNC / VINVALL`	`vPEID, …`	GICv4 virtual equivalents. VMOVP carries a SequenceNumber to coordinate across multiple ITSs. v4.1 adds VSGI and INVDB.

08 · Architecture

Virtualization: v3 list registers to v4 direct injection



v4.0

vLPIs injected directly. Scheduling a vPE writes `GICR_VPROPBASER` / `VPENDBASER`. Doorbells are per-interrupt (Dbell_pINTID in VMAPTI).

v4.1

Direct vSGIs (`VSGI` command, `GICD_CTLR.nASSGIreq`). Per-vPE default doorbell, `INVDB`, vPE table shared and addressed through the RD (`GICR_VPROPBASER` becomes the vPE table pointer).

v4.2

Virtual interrupts can carry the NMI attribute, so a VM gets NMIs without trapping.

Classic bug area. Descheduling a vPE (VPENDBASER.Valid 1→0) while a vLPI is in flight. The RD must finish writing the pending table (the `Dirty` bit) before the hypervisor reuses it, and the interrupt must either be in the VPT or raise a doorbell, never neither.

09 · Design

Power management and the stream interface

Redistributor power handshake (GICR_WAKER)

1. Before powering down a PE, software sets `ProcessorSleep` =1.
2. The RD quiesces the CPU interface over the stream (Quiesce / Quiesce Acknowledge).
3. The RD sets `ChildrenAsleep` =1. Software polls it, then removes power.
4. While asleep, an interrupt targeted at this PE raises a wake request (for example `WakeRequest` to the power controller) instead of IRQ.
5. Wake-up: clear `ProcessorSleep`, poll `ChildrenAsleep` =0.

Asleep PEs drop out of 1-of-N SPI selection. GICR_PWRR (IMPLEMENTATION DEFINED in some designs) can power the RD itself.

GIC Stream Protocol (Appendix A)

AXI4-Stream style packets between the Interrupt Routing Infrastructure (IRI: Distributor + RDs) and the CPU interface (ICC).

IRI → CPU IF	CPU IF → IRI
Set (offer HPPI)	Activate (on IAR read)
Clear (withdraw)	Clear Acknowledge
Activate Acknowledge	Deactivate (EOI/DIR)
Deactivate Acknowledge	Release (decline offered int)
Generate SGI Ack	Generate SGI
Quiesce	Quiesce Acknowledge
Upstream Control Ack	Upstream Control
Downstream Control	Downstream Control Ack
VSet / VClear (v4.1+)	—

Key race: the RD sends Clear while the CPU is reading IAR. The CPU interface must either Activate the INTID the RD still owns or return 1023. A Set it cannot accept must get a Release.

Register map at a glance

BLOCK	MUST-KNOW REGISTERS	INTERVIEW NOTE
GICD_	CTLR TYPER IIDR IGROUPR IGRPMODR ISENABLER ICENABLER ISPENDR ICPENDR ISACTIVER ICACTIVER IPRIORITYR ICFGR IROUTER SETSPI_NSR NSACR	Set/clear register pairs give atomic per-bit updates without read-modify-write. `CTLR.RWP` must be polled after disabling groups or ARE changes.
GICR_ RD_base	CTLR TYPER WAKER PROPBASER PENDBASER SETLPIR CLRLPIR INVLPIR SYNCR VPROPBASER VPENDBASER	`TYPER` gives affinity and `Last`; software walks frames to find its RD. `CTLR.EnableLPIs` may be one-way (RES1 once set) in some implementations.
GICR_ SGI_base	IGROUPR0 IGRPMODR0 ISENABLER0 ICENABLER0 IPRIORITYR0-7 ICFGR0/1 NSACR	The per-PE copy of what GICD holds for SPIs; SGIs are always edge.
GITS_	CTLR TYPER IIDR BASER<n> CBASER CWRITER CREADR TRANSLATER	`BASER` types: Device, Collection, vPE. Indirect (two-level) tables save memory for sparse DeviceIDs.
ICC_	SRE PMR IAR0/1 NMIAR1 EOIR0/1 DIR HPPIR0/1 BPR0/1 RPR CTLR IGRPEN0/1 AP0R/AP1R SGI0R SGI1R ASGI1R	`ICC_SRE_ELx.SRE`=1 enables system-register access. Self-synchronization: a PMR write needs no ISB to take effect for signaling, but software uses one for ordering.
ICH_ / ICV_	HCR VTR VMCR LR<n> AP0R/AP1R MISR EISR ELRSR · ICV_ mirrors ICC_	`HCR_EL2.IMO/FMO`=1 redirects EL1 ICC_ accesses to the ICV_ virtual interface. `VTR.ListRegs` gives the number of LRs (up to 16).

11 · Microarchitecture

How it is actually built

The spec says what must be observable. Silicon makes choices that create the bugs you will be asked about.

HPPI selection

Priority-select tree across up to ~1000 SPIs plus per-PE PPIs and LPI cache entries. Often pipelined or iterative, so the "current best" can be stale by a few cycles. Ties break on IMPLEMENTATION DEFINED order, usually the lowest INTID.

LPI caching

RDs hold a small cache of pending LPIs and config bytes. Fill from memory on miss, evict back to the pending table. Coherency with software edits relies on INV/INVALL/SYNC. This is where ordering bugs live.

ITS caching

Device, ITT and collection entries are cached. Commands must invalidate them in order. Command processing overlaps translation, so a MOVI racing an incoming MSI is the hardest scenario.

Topology

Large SoCs use multi-chip (GICD_CHIPR / multi-chip operation) or distribute the Distributor across clusters. RDs often sit next to the cluster, linked by the stream interface or a NoC.

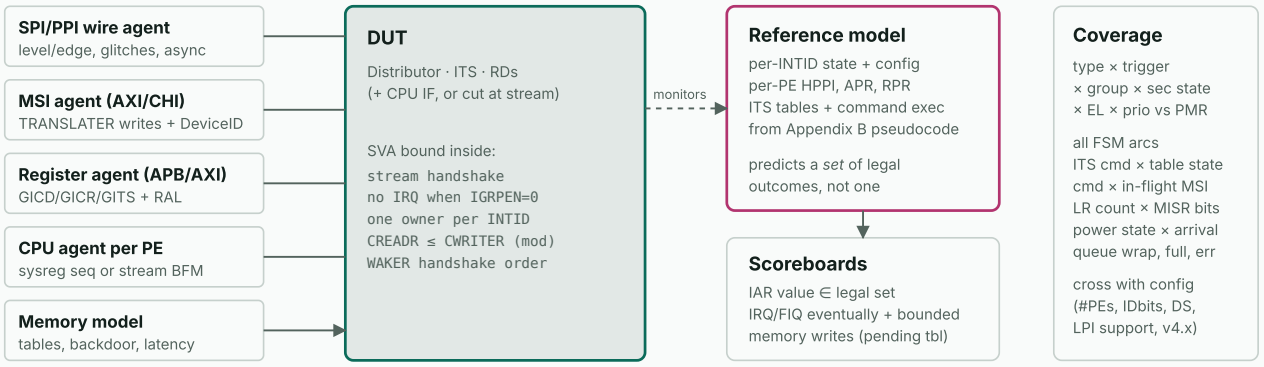
Clock and reset domains

Wired SPIs cross into the GIC clock (synchronizers, glitch filters for level lines). CPU interface runs at core clock. RD and Distributor can be power-gated independently.

Errors and RAS

Chapter 14 covers system error reporting: ITS command errors (an error code in `GITS_CREADR`/IMPLEMENTATION DEFINED syndrome), bad table walks, ECC on caches. Errors can stall the queue or be signaled as SError/an interrupt.

How a senior DV engineer attacks the GIC



Run the same tests on several configurations; most GIC escapes are config-specific (DS=1, no ITS, 1 PE, max PEs, non-power-of-two IDbits).

Principles to say out loud

- **Model the set of legal outcomes.** The spec leaves timing open ("eventually", IMPLEMENTATION DEFINED 1-of-N, either old or new target). A scoreboard that predicts one exact answer floods you with false fails.
- **Check at architectural observation points:** IAR/HPPIR reads, IRQ/FIQ pins, register readback, memory written by the GIC. Do not check internal timing.
- **Separate signaling from acknowledge.** Signaling may be stale; the IAR value must be legal at the moment of the read.
- **Layer the TB:** block level (ITS alone with a stream/RD stub; CPU IF alone with a stream BFM), then GIC subsystem, then SoC with real cores running a bare-metal or Linux driver.
- **Use the pseudocode.** Appendix B gives functions such as priority and group-masking checks; transcribe them into the model to avoid reinterpretation bugs.

Test plan skeleton

- Reset values and register access rules (RAZ/WI by Security state, RES0/RES1, width, NSACR).
- Single interrupt, every type × group × trigger × target, end to end.
- Priority ladder: N pending, check strict order and BPR preemption; nested ack/EOI up to max APR depth.
- Masking: PMR sweeps, group enable toggles mid-flight, RPR checks.
- Routing: IROUTER sweeps, 1-of-N fairness and exclusion, SGI broadcast and RS.
- LPI: every ITS command, table formats (flat/indirect), INV/SYNC ordering, queue wrap and stall.
- Virtualization: LR life cycle, HW=1 linkage, maintenance interrupts, vPE schedule/deschedule storms, doorbells, VMOVP across ITSs.
- Power: WAKER around every other test; wake on SPI/LPI/SGI.
- Random stress: concurrent config writes + traffic, with the model as the oracle.
- Linux/firmware boot (irq-gic-v3, TF-A) as the final system test.

13 · Verification mindset

Bug hunt list

Have three or four of these ready as "a bug I found / would target" stories. Each is a real class of escape.

AREA	SCENARIO	EXPECTED BEHAVIOUR TO CHECK
Lifecycle	Level SPI deasserts after ack, before EOI	State goes Active (not A&P); no re-fire after deactivate.
Lifecycle	Edge arrives exactly as IAR is read	Ends Active & Pending, fires once more after deactivate; never lost.
Masking	PMR raised while IRQ is asserted	IRQ drops; an IAR read in the window returns 1023 or the old INTID (both legal), never a masked one if the drop is visible.
Config race	ICENABLER write while the SPI is being forwarded	After RWP=0, no new signaling of that SPI; already-acked one still completes.
Priority	EOI with EOImode=1, then DIR out of order	DIR works in any order; EOI drops the highest active priority only.
Priority	EOI of an INTID that is not the highest active	UNPREDICTABLE in spec; DUT must not hang or corrupt other INTIDs (robustness).
Routing	1-of-N SPI, the chosen PE goes to sleep	Re-selected to another participating PE or wake request; never stuck.
ITS	MOVI while the LPI is pending, then MSI arrives	Exactly one pending instance, on the new RD after SYNC.
ITS	Config byte edited in memory without INV	Old or new value acceptable until INV/SYNC; must be new afterwards.
ITS	Command queue wraps; CWRITER = CREADR − 1 (full)	No overrun; correct wrap at CBASER.Size.
ITS	MAPD V=0 with mapped events pending	No translation afterwards; behaviour of existing pending state per spec, no stale cache hits.
LPI	EnableLPIs cleared, pending table written back, re-enabled	Pending state survives exactly; Dirty/RWP polled.
Virtual	LR with HW=1 and pINTID, VM EOIs	Physical INTID deactivated once; no double deactivate.
Virtual	vPE descheduled while vLPI arrives	Interrupt in VPT and/or doorbell; never dropped.
Security	NS write to a Secure Group register/field	RAZ/WI; no side effects; DS=1 behaves differently.

AREA	SCENARIO	EXPECTED BEHAVIOUR TO CHECK
NMI	NMI pending with PMR=0 and ALLINT=0	Still signaled; IAR1 returns 1022 when SCTLR.NMI=1.

14 · Interview

Questions you should expect

Open all answers | Close all

– **Walk me through what happens from a UART raising its line to the handler running.** core

The SPI line asserts; the Distributor marks it pending (level). If enabled, its group is enabled and IROUTER targets PE X, the Distributor forwards it to RD X. The RD picks it as HPPI if it beats other candidates and sends a Set to the CPU interface. The CPU interface checks group enable, PMR and running priority, then asserts IRQ or FIQ according to group and current Security state. The core takes the exception, reads `ICC_IAR1_EL1` (INTID returned, state becomes Active, running priority raised), services and clears the UART, writes `ICC_EOIR1_EL1` (priority drop + deactivate with EOImode=0).

– **Why did GICv3 move the CPU interface to system registers?** core

Lower latency than an MMIO round trip on the interconnect, natural per-PE banking, cleaner virtualization by redirecting EL1 accesses to ICV_ registers through HCR_EL2.IMO/FMO, and scalability past 8 cores with affinity routing.

– **Difference between SPI, PPI, SGI and LPI, and who owns state for each?** core

SGI 0–15 and PPI 16–31 (plus EPPI) are private to a PE; state in the Redistributor's SGI_base frame. SPI 32–1019 (plus ESPI) are shared; state in the Distributor and routed by IROUTER. LPIs (8192+) are message based, edge-like with no active state; config in a memory table, pending in a per-RD memory table cached in the RD.

– **An interrupt fires as FIQ when you expected IRQ. What do you check?** debug

Its group (IGROUPR/IGRPMODR) versus the PE's current Security state and EL. Group 0 is always FIQ; a Group 1 interrupt for the other Security state, or anything at EL3, is FIQ. Then check DS, ARE settings and whether SCR_EL3.FIQ/IRQ routing matches what firmware intended.

– Explain PMR, BPR and running priority with an example. `core`

PMR=0xF0: only priorities below 0xF0 can signal. BPR0=2 → group priority is bits [7:3]. Handling 0x40 sets running priority 0x40. A new 0x38 has group priority 0x38 < 0x40, so it preempts. A new 0x44 (group priority 0x40) does not preempt, even though it passes PMR; it waits until the EOI drops the running priority.

– What is EOImode=1 for, and how does a hypervisor use it? `deep`

It splits priority drop (EOIR) from deactivation (DIR). The hypervisor acks a physical interrupt, drops priority so the host can take other interrupts, and leaves it Active. It writes an LR with HW=1 and the pINTID. When the guest EOIs the virtual interrupt, hardware deactivates the physical one, so no trap is needed.

– How does the ITS translate an MSI? What are its tables? `core`

The write to GITS_TRANSLATER supplies EventID (data) and DeviceID (sideband, for example from the SMMU or PCIe RID). Device table[DeviceID] → ITT; ITT[EventID] → INTID + ICID; Collection table[ICID] → target RD. The RD sets the LPI pending bit. Tables are allocated by software via GITS_BASERn and populated by commands (MAPD, MAPTI, MAPC).

– Software changed an LPI's priority in memory, but nothing happened. Why? `debug`

The RD caches LPI configuration. Software must issue INV (DeviceID, EventID) or INVALL (ICID) followed by SYNC, or write GICR_INVLPIR, before the change is guaranteed visible. In DV, the model should allow old or new values until the invalidation completes.

– How would you verify MOVI? `dv`

Directed: map an event to collection A, make it pending (INT), MOVI to collection B, SYNC, check pending only at RD B and exactly one acknowledge. Random: inject MSIs on the same event concurrently with MOVI at varying offsets, with B's PE asleep or with LPIs masked. Checker: conservation (no lost or duplicated interrupt per event across the window), final location after SYNC. Coverage: MOVI × {inactive, pending, cached, uncached} × {MSI before, during, after}.

– How do you build a scoreboard when the spec allows non-deterministic outcomes? dv

The model keeps a set of legal candidates and a window. For each IAR read, the returned INTID must be any interrupt that was the legal HPPI at some point between the last config change and the read, or 1023 if none was guaranteed. Pin checks use eventual assertions with a bounded timeout. Routing with IRM=1 checks membership in the participating set. This avoids both false fails and missed bugs.

– What is a maintenance interrupt and which conditions raise it? deep

A PPI (INTID 25 by convention) to the hypervisor, controlled by ICH_HCR_EL2 and reported in ICH_MISR_EL2: underflow (UIE, LRs nearly empty so refill), no pending (NPIE), LR entry not present on EOI (LRENPIE, counted in EOIcount), and group enable/disable transitions (VGrp0E/D, VGrp1E/D). They let the hypervisor manage more virtual interrupts than it has LRs.

– GICv4 vs v4.1: what problem does each solve? deep

v4.0 removes the hypervisor from vLPI delivery: the ITS writes the vPE's virtual pending table and the RD signals the resident vPE, or rings a doorbell if it is not resident. v4.1 adds direct vSGIs (so IPIs inside a VM need no trap), a per-vPE default doorbell and reorganized vPE tables that RDs can access directly.

– How do you safely power down a core's Redistributor? core

Disable or migrate interrupts targeting it (MOVALL for LPIs, retarget SPIs), set GICR_WAKER.ProcessorSleep, wait for ChildrenAsleep, then power off. Interrupts that arrive later generate a wake request. On power-up, clear ProcessorSleep and wait for ChildrenAsleep=0 before enabling groups.

– Where would you cut the design to verify it at block level? dv

At the GIC Stream Protocol between IRI and CPU interface: a stream BFM can act as a CPU interface to test Distributor + RDs + ITS at high throughput, and another can act as the IRI to test the CPU interface with real core sysreg sequences. The ITS can also be isolated behind its RD-facing interface. Protocol SVA on the stream catches handshake bugs early.

– What coverage would make you confident to sign off? dv

Functional: every state transition per type, the full IRQ/FIQ signaling matrix, priority/BPR/PMR crosses, every ITS command against every relevant table state, concurrency crosses (command × in-flight MSI, config write × forwarding), power-state × arrival, LR/MISR combinations, every legal configuration parameter. Plus code and toggle coverage, assertion coverage on stream and WAKER, zero unexplained scoreboard waivers and a clean Linux boot with interrupt stress.

– How does a 1-of-N SPI pick a target and what can go wrong? deep

The choice among participating PEs is IMPLEMENTATION DEFINED. Common bugs: picking a PE whose group is disabled or which is asleep, starving one PE, or two PEs both acknowledging the same SPI. The checker asserts exactly one Activate per pending instance and membership in the participating set.

15 · Interview

The 60-second pitch

> "GICv3 has four blocks. The Distributor owns shared SPIs. A Redistributor per core owns private SGIs and PPIs and the memory-backed LPIs. The ITS translates device MSIs into LPIs using Device, ITT and Collection tables driven by a command queue. The CPU interface is in system registers and applies group, PMR and running-priority checks before raising IRQ or FIQ, based on the interrupt's group and the core's Security state. Software acknowledges through IAR and ends with EOIR, optionally split from DIR. Virtualization adds list registers in v3 and direct vLPI and vSGI injection in v4. When I verify it, I model the set of outcomes the spec allows instead of one exact answer, check at architectural observation points, cut the design at the stream protocol for block-level throughput, and spend my stress effort on races: config versus traffic, ITS commands versus in-flight MSIs, and power transitions versus arrivals."

Built from Arm IHI 0069H.b (GICv3.3 / GICv4.2, April 2024). Check exact field encodings in Chapter 12 before quoting bit positions in an interview.